

SQL vs PySpark: Complete Syntax Comparison Guide

Introduction

This comprehensive guide provides a detailed comparison between SQL and PySpark syntax for common data operations. SQL is a declarative language ideal for structured queries on traditional databases, while PySpark is a procedural API designed for distributed big data processing using Apache Spark[1].

Understanding the equivalence between SQL and PySpark is crucial for data engineers and analysts working in hybrid environments where both technologies are used. SQL provides a declarative way to interact with data, whereas PySpark leverages Resilient Distributed Datasets (RDDs) and DataFrames to perform transformations and actions efficiently across distributed systems[7].

Both approaches are powerful, but they serve different use cases. SQL excels in readability and simplicity for analysts familiar with relational databases, while PySpark provides programmatic control and scalability for large-scale data processing across distributed clusters[2].

Key Differences:

- **SQL** – Declarative, database-centric, simple syntax for structured queries
- **PySpark** – Procedural, Python-based, distributed computing framework
- **Performance** – Both use Spark's Catalyst Optimizer internally, offering similar performance[3]
- **Interoperability** – PySpark can execute SQL queries and vice versa through temporary views

Data Types Reference

Understanding data type equivalence is fundamental when translating SQL schemas to PySpark.

SQL Data Type	PySpark Equivalent
INT	IntegerType()
BIGINT	LongType()
FLOAT	FloatType()
DOUBLE	DoubleType()
CHAR(n) / VARCHAR(n)	StringType()

DATE	DateType()
TIMESTAMP	TimestampType()

Table 1: SQL to PySpark data type mapping

Database and Table Operations

Database Management

Concept	SQL Query	PySpark Equivalent
Create Database	CREATE DATABASE db_name;	spark.sql("CREATE DATABASE db_name")
Use Database	USE db_name;	spark.catalog.setCurrentDatabase("db_name")
Drop Database	DROP DATABASE db_name;	spark.sql("DROP DATABASE db_name")
Show Databases	SHOW DATABASES;	spark.sql("SHOW DATABASES").show()

Table 2: Database management operations

Table Management

Concept	SQL Query	PySpark Equivalent
Create Table	CREATE TABLE table_name (col1 INT, col2 STRING);	df.write.format("parquet").saveAsTable("table_name")
Drop Table	DROP TABLE table_name;	spark.sql("DROP TABLE IF EXISTS table_name")
Truncate Table	TRUNCATE TABLE table_name;	spark.sql("TRUNCATE TABLE table_name")
Describe Table	DESCRIBE TABLE table_name;	df.printSchema()

Show Tables	SHOW TABLES;	spark.sql("SHOW TABLES").show()
-------------	--------------	---------------------------------

Table 3: Table management operations

Table Alterations

Concept	SQL Query	PySpark Equivalent
Add Column	ALTER TABLE table_name ADD COLUMN col3 STRING;	df.withColumn("col3", lit(None).cast("string"))
Rename Column	ALTER TABLE table_name RENAME COLUMN old_name TO new_name;	df.withColumnRenamed("old_name", "new_name")
Drop Column	ALTER TABLE table_name DROP COLUMN col3;	df.drop("col3")

Table 4: Table alteration operations

Partitioning and Bucketing

Concept	SQL Query	PySpark Equivalent
Create Partitioned Table	CREATE TABLE table_name (col1 INT, col2 STRING) PARTITIONED BY (col3 STRING);	df.write.partitionBy("col3").format("parquet").saveAsTable("table_name")

Insert into Partitioned Table	<pre>INSERT INTO table_name PARTITION (col3='value') SELECT col1, col2 FROM source_table;</pre>	<pre>df.write.mode("append").partitionBy("col3").saveAsTable("table_name")</pre>
Create Bucketed Table	<pre>CREATE TABLE table_name (col1 INT, col2 STRING) CLUSTERED BY (col1) INTO 10 BUCKETS;</pre>	<pre>df.write.bucketBy(10, "col1").saveAsTable("table_name")</pre>

Table 5: Partitioning and bucketing strategies

Views Management

Concept	SQL Query	PySpark Equivalent
Create View	<pre>CREATE VIEW view_name AS SELECT * FROM table_name;</pre>	<pre>df.createOrReplaceTempView("view_name")</pre>
Drop View	<pre>DROP VIEW view_name;</pre>	<pre>spark.sql("DROP VIEW IF EXISTS view_name")</pre>
Create Global View	<pre>CREATE GLOBAL VIEW view_name AS SELECT * FROM table_name;</pre>	<pre>df.createGlobalTempView("view_name")</pre>
Show Views	<pre>SHOW VIEWS;</pre>	<pre>spark.sql("SHOW VIEWS").show()</pre>

Table 6: View creation and management

Schema Management

Concept	SQL Query	PySpark Equivalent
Define Schema Manually	CREATE TABLE table_name (col1 INT, col2 STRING, col3 DATE);	from pyspark.sql.types import StructType, StructField, IntegerType, StringType, DateType; schema = StructType([StructField("col1", IntegerType(), True), StructField("col2", StringType(), True), StructField("col3", DateType(), True)])
Check Schema	DESCRIBE TABLE table_name;	df.printSchema()
Change Column Data Type	ALTER TABLE table_name ALTER COLUMN col1 TYPE BIGINT;	df.withColumn("col1", col("col1").cast("bigint"))

Table 7: Schema definition and management

File-Based Table Operations

Concept	SQL Query	PySpark Equivalent
Save as Parquet	N/A (Implicit in Hive)	df.write.format("parquet").save("path/to/parquet")
Save as Delta Table	CREATE TABLE table_name USING DELTA LOCATION 'path';	df.write.format("delta").save("path/to/delta")
Save as CSV	N/A	df.write.format("csv").option("header", True).save("path/to/csv")
Save as JSON	N/A	df.write.format("json").save("path/to/json")

Save as ORC	N/A	<code>df.write.format("orc").save("path/to/orc")</code>
----------------	-----	---

Table 8: File format operations

1. Basic Data Selection

SELECT All Columns

SQL:

```
SELECT * FROM employees;
```

PySpark DataFrame API:

```
df = spark.table("employees")
df.show()
```

PySpark SQL API:

```
spark.sql("SELECT * FROM employees").show()
```

SELECT Specific Columns

SQL:

```
SELECT employeeName, employeeSurname, employeeTitle
FROM employees;
```

PySpark DataFrame API:

```
df = spark.table("employees")
df.select("employeeName", "employeeSurname", "employeeTitle").show()
```

PySpark Alternative Syntax:

```
df.select(df.employeeName, df.employeeSurname, df.employeeTitle).show()
```

Column Aliasing

SQL:

```
SELECT employeeName AS name,
employeeSurname AS surname
FROM employees;
```

PySpark DataFrame API:

```
df.select(
df.employeeName.alias("name"),
df.employeeSurname.alias("surname")
).show()
```

2. Filtering Data (WHERE Clause)

Basic Filtering

SQL:

```
SELECT name, age  
FROM employees  
WHERE age > 30;
```

PySpark DataFrame API:

```
df.filter(df.age > 30).select("name", "age").show()
```

PySpark Alternative (where method):

```
df.where(df.age > 30).select("name", "age").show()
```

PySpark with SQL String:

```
df.where("age > 30").select("name", "age").show()
```

Multiple Conditions (AND)

SQL:

```
SELECT *  
FROM employees  
WHERE age > 30 AND department = 'Sales';
```

PySpark DataFrame API:

```
df.filter((df.age > 30) & (df.department == 'Sales')).show()
```

PySpark SQL String:

```
df.filter("age > 30 AND department = 'Sales'").show()
```

Multiple Conditions (OR)

SQL:

```
SELECT *  
FROM employees  
WHERE department = 'Sales' OR department = 'Marketing';
```

PySpark DataFrame API:

```
df.filter((df.department == 'Sales') | (df.department == 'Marketing')).show()
```

IN Operator

SQL:

```
SELECT *  
FROM employees  
WHERE department IN ('Sales', 'Marketing', 'IT');
```

PySpark DataFrame API:

```
from pyspark.sql.functions import col
```

```
df.filter(col("department").isin(['Sales', 'Marketing', 'IT'])).show()
```

NULL Handling

SQL:

```
SELECT *  
FROM employees  
WHERE salary IS NOT NULL AND age > 25;
```

PySpark DataFrame API:

```
df.filter((df.salary.isNotNull()) & (df.age > 25)).show()
```

PySpark SQL String:

```
df.where("salary IS NOT NULL AND age > 25").show()
```

3. Aggregation and Grouping

Basic Aggregation

SQL:

```
SELECT COUNT(*) AS total_employees,  
AVG(salary) AS avg_salary,  
MAX(salary) AS max_salary,  
MIN(salary) AS min_salary  
FROM employees;
```

PySpark DataFrame API:

```
from pyspark.sql.functions import count, avg, max, min
```

```
df.agg(  
count("*").alias("total_employees"),  
avg("salary").alias("avg_salary"),  
max("salary").alias("max_salary"),  
min("salary").alias("min_salary")  
)show()
```

GROUP BY

SQL:

```
SELECT department, AVG(salary) AS avg_salary  
FROM employees  
GROUP BY department;
```

PySpark DataFrame API:

```
from pyspark.sql.functions import avg
```

```
df.groupBy("department").agg(avg("salary").alias("avg_salary")).show()
```

GROUP BY with Multiple Aggregations

SQL:

```
SELECT department,  
COUNT(*) AS employee_count,  
AVG(salary) AS avg_salary,  
MAX(salary) AS max_salary  
FROM employees  
GROUP BY department;
```

PySpark DataFrame API:

```
from pyspark.sql.functions import count, avg, max
```

```
df.groupBy("department").agg(  
count("*").alias("employee_count"),  
avg("salary").alias("avg_salary"),  
max("salary").alias("max_salary")  
)>show()
```

HAVING Clause

SQL:

```
SELECT department, AVG(salary) AS avg_salary  
FROM employees  
GROUP BY department  
HAVING AVG(salary) > 50000;
```

PySpark DataFrame API:

```
from pyspark.sql.functions import avg
```

```
df.groupBy("department")  
.agg(avg("salary").alias("avg_salary"))  
.filter("avg_salary > 50000")  
>show()
```

4. Sorting Data

ORDER BY (Ascending)

SQL:

```
SELECT *  
FROM employees  
ORDER BY salary ASC;
```

PySpark DataFrame API:

```
df.orderBy("salary").show()
```

OR

```
df.orderBy(df.salary.asc()).show()
```

ORDER BY (Descending)

SQL:

```
SELECT *  
FROM employees  
ORDER BY salary DESC;
```

PySpark DataFrame API:

```
df.orderBy(df.salary.desc()).show()
```

Multiple Column Sorting

SQL:

```
SELECT *  
FROM employees  
ORDER BY department ASC, salary DESC;
```

PySpark DataFrame API:

```
df.orderBy(df.department.asc(), df.salary.desc()).show()
```

5. Joins

INNER JOIN

SQL:

```
SELECT a.name, b.department  
FROM employees a  
INNER JOIN departments b ON a.dept_id = b.dept_id;
```

PySpark DataFrame API:

```
df1.join(df2, df1.dept_id == df2.dept_id, "inner")  
.select(df1.name, df2.department)  
.show()
```

LEFT JOIN

SQL:

```
SELECT a.name, b.department  
FROM employees a  
LEFT JOIN departments b ON a.dept_id = b.dept_id;
```

PySpark DataFrame API:

```
df1.join(df2, df1.dept_id == df2.dept_id, "left")  
.select(df1.name, df2.department)  
.show()
```

RIGHT JOIN

SQL:

```
SELECT a.name, b.department
FROM employees a
RIGHT JOIN departments b ON a.dept_id = b.dept_id;
```

PySpark DataFrame API:

```
df1.join(df2, df1.dept_id == df2.dept_id, "right")
.select(df1.name, df2.department)
.show()
```

FULL OUTER JOIN

SQL:

```
SELECT a.name, b.department
FROM employees a
FULL OUTER JOIN departments b ON a.dept_id = b.dept_id;
```

PySpark DataFrame API:

```
df1.join(df2, df1.dept_id == df2.dept_id, "outer")
.select(df1.name, df2.department)
.show()
```

Multiple Join Conditions

SQL:

```
SELECT a.name, b.department
FROM employees a
JOIN departments b
ON a.dept_id = b.dept_id
AND a.location = b.location;
```

PySpark DataFrame API:

```
df1.join(df2,
(df1.dept_id == df2.dept_id) & (df1.location == df2.location),
"inner"
).select(df1.name, df2.department).show()
```

6. String Operations

CONCAT

SQL:

```
SELECT CONCAT(firstName, ' ', lastName) AS fullName
FROM employees;
```


PySpark DataFrame API:

```
from pyspark.sql.functions import concat, lit
```

```
df.select(concat(df.firstName, lit(' '), df.lastName).alias("fullName")).show()
```

UPPER and LOWER

SQL:

```
SELECT UPPER(name) AS upper_name,  
LOWER(name) AS lower_name  
FROM employees;
```

PySpark DataFrame API:

```
from pyspark.sql.functions import upper, lower
```

```
df.select(  
    upper(df.name).alias("upper_name"),  
    lower(df.name).alias("lower_name")  
)show()
```

SUBSTRING

SQL:

```
SELECT SUBSTRING(name, 1, 3) AS short_name  
FROM employees;
```

PySpark DataFrame API:

```
from pyspark.sql.functions import substring
```

```
df.select(substring(df.name, 1, 3).alias("short_name")).show()
```

LIKE Pattern Matching

SQL:

```
SELECT *  
FROM employees  
WHERE name LIKE 'John%';
```

PySpark DataFrame API:

```
df.filter(df.name.like('John%')).show()
```

Additional String Operations

Concept	SQL Query	PySpark Equivalent
String Length	SELECT LEN(column) FROM table;	df.select(length(col("column")))

Trim String	<pre>SELECT TRIM(column) FROM table;</pre>	<pre>df.select(trim(col("column")))</pre>
-------------	--	---

Table 9: Additional string manipulation functions

7. Date and Time Operations

Current Date and Timestamp

SQL:

```
SELECT CURRENT_DATE() AS today,
CURRENT_TIMESTAMP() AS now;
```

PySpark DataFrame API:

```
from pyspark.sql.functions import current_date, current_timestamp
```

```
df.select(
    current_date().alias("today"),
    current_timestamp().alias("now")
).show()
```

Date Extraction

SQL:

```
SELECT YEAR(hire_date) AS year,
MONTH(hire_date) AS month,
DAY(hire_date) AS day
FROM employees;
```

PySpark DataFrame API:

```
from pyspark.sql.functions import year, month, dayofmonth
```

```
df.select(
    year(df.hire_date).alias("year"),
    month(df.hire_date).alias("month"),
    dayofmonth(df.hire_date).alias("day")
).show()
```

Date Difference

SQL:

```
SELECT DATEDIFF(CURRENT_DATE(), hire_date) AS days_employed
FROM employees;
```

PySpark DataFrame API:

```
from pyspark.sql.functions import datediff, current_date
```

```
df.select(
datediff(current_date(), df.hire_date).alias("days_employed")
).show()
```

Date Formatting

SQL:

```
SELECT DATE_FORMAT(hire_date, 'yyyy-MM-dd') AS formatted_date
FROM employees;
```

PySpark DataFrame API:

```
from pyspark.sql.functions import date_format
```

```
df.select(
date_format(df.hire_date, 'yyyy-MM-dd').alias("formatted_date")
).show()
```

Additional Date and Time Functions

Concept	SQL Query	PySpark Equivalent
Current Date	SELECT CURDATE();	df.select(current_date())
Current Timestamp	SELECT NOW();	df.select(current_timestamp())
CAST/CONVERT	SELECT CAST(column AS datatype) FROM table;	df.select(col("column").cast("datatype"))

Table 10: Date and time manipulation functions

8. Window Functions

ROW_NUMBER

SQL:

```
SELECT name, salary,
ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS rank
FROM employees;
```

PySpark DataFrame API:

```
from pyspark.sql.window import Window
from pyspark.sql.functions import row_number
```



```
window_spec = Window.partitionBy("department").orderBy(df.salary.desc())
```

```
df.select(  
df.name,  
df.salary,  
row_number().over(window_spec).alias("rank")  
)show()
```

RANK and DENSE_RANK

SQL:

```
SELECT name, salary,  
RANK() OVER (ORDER BY salary DESC) AS rank,  
DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rank  
FROM employees;
```

PySpark DataFrame API:

```
from pyspark.sql.window import Window  
from pyspark.sql.functions import rank, dense_rank
```

```
window_spec = Window.orderBy(df.salary.desc())
```

```
df.select(  
df.name,  
df.salary,  
rank().over(window_spec).alias("rank"),  
dense_rank().over(window_spec).alias("dense_rank")  
)show()
```

LAG and LEAD

SQL:

```
SELECT name, salary,  
LAG(salary, 1) OVER (ORDER BY hire_date) AS prev_salary,  
LEAD(salary, 1) OVER (ORDER BY hire_date) AS next_salary  
FROM employees;
```

PySpark DataFrame API:

```
from pyspark.sql.window import Window  
from pyspark.sql.functions import lag, lead
```

```
window_spec = Window.orderBy("hire_date")
```

```
df.select(  
df.name,  
df.salary,  
lag(df.salary, 1).over(window_spec).alias("prev_salary"),  
lead(df.salary, 1).over(window_spec).alias("next_salary")  
)show()
```

Comprehensive Window Functions Reference

Concept	SQL Query	PySpark Equivalent
RANK()	<pre>SELECT column, RANK() OVER (PARTITION BY col2 ORDER BY column) FROM table;</pre>	<pre>df.withColumn("rank", rank().over(Window.partitionBy("col2").ord erBy("column")))</pre>
DENSE_RANK()	<pre>SELECT column, DENSE_R ANK() OVER (PARTITION BY col2 ORDER BY column) FROM table;</pre>	<pre>df.withColumn("dense_rank", dense_rank().over(Window.partitionBy("col2").ord erBy("column")))</pre>
ROW_NUMBER()	<pre>SELECT column, ROW_NUM BER() OVER (PARTITION BY col2 ORDER BY column) FROM table;</pre>	<pre>df.withColumn("row_number", row_number().over(Window.partitionBy("col2").or derBy("column")))</pre>
LEAD()	<pre>SELECT column, LEAD(column, 1) OVER (PARTITION BY col2 ORDER BY column) FROM table;</pre>	<pre>df.withColumn("lead_value", lead("column", 1).over(Window.partitionBy("col2").orderBy("column")))</pre>

LAG()	<pre>SELECT column, LAG(column , 1) OVER (PARTITION BY col2 ORDER BY column) FROM table;</pre>	<pre>df.withColumn("lag_value", lag("column", 1).over(Window.partitionBy("col2").orderBy("column")))</pre>
-------	--	--

Table 11: Comprehensive window functions reference

9. NULL Handling

COALESCE

SQL:

```
SELECT name, COALESCE(salary, 0) AS salary
FROM employees;
```

PySpark DataFrame API:

from pyspark.sql.functions import coalesce, lit

```
df.select(
df.name,
coalesce(df.salary, lit(0)).alias("salary")
).show()
```

Fill NULL Values

SQL:

```
SELECT IFNULL(salary, 0) AS salary
FROM employees;
```

PySpark DataFrame API:

```
df.fillna({'salary': 0}).show()
```

Drop NULL Values

SQL:

```
SELECT *
FROM employees
WHERE salary IS NOT NULL;
```

PySpark DataFrame API:

```
df.dropna(subset=['salary']).show()
```


10. Conditional Logic

CASE WHEN

SQL:

```
SELECT name, salary,  
CASE  
WHEN salary > 80000 THEN 'High'  
WHEN salary > 50000 THEN 'Medium'  
ELSE 'Low'  
END AS salary_category  
FROM employees;
```

PySpark DataFrame API:

```
from pyspark.sql.functions import when
```

```
df.select(  
df.name,  
df.salary,  
when(df.salary > 80000, "High")  
.when(df.salary > 50000, "Medium")  
.otherwise("Low")  
.alias("salary_category")  
)show()
```

Additional Conditional Operations

Concept	SQL Query	PySpark Equivalent
IF (Conditional Logic)	SELECT IF(condition, value1, value2) FROM table;	df.select(when(condition, value1).otherwise(value2))

Table 12: Conditional logic operations

Logical Operators

Concept	SQL Query	PySpark Equivalent
AND / OR	SELECT * FROM table WHERE column1 = value AND column2 > value;	df.filter((col("column1") == value) & (col("column2") > value))

IS NULL / IS NOT NULL	SELECT * FROM table WHERE column IS NULL;	df.filter(col("column").isNull())
LIKE	SELECT * FROM table WHERE column LIKE 'value%';	df.filter(col("column").like("value%"))
BETWEEN	SELECT * FROM table WHERE column BETWEEN value1 AND value2;	df.filter((col("column") >= value1) & (col("column") <= value2))

Table 13: Logical operators comparison

11. Subqueries and CTEs

Subquery in WHERE Clause

SQL:

```
SELECT *
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

PySpark DataFrame API:

```
from pyspark.sql.functions import avg

avg_salary = df.select(avg("salary")).collect()[0][0]
df.filter(df.salary > avg_salary).show()
```

Common Table Expression (CTE)

SQL:

```
WITH high_earners AS (
SELECT * FROM employees WHERE salary > 80000
)
SELECT department, COUNT(*) AS count
FROM high_earners
GROUP BY department;
```

PySpark DataFrame API:

```
high_earners = df.filter(df.salary > 80000)
high_earners.groupBy("department").count().show()
```

PySpark SQL with Temporary View:

```
df.createOrReplaceTempView("employees")

result = spark.sql("""
WITH high_earners AS (
SELECT * FROM employees WHERE salary > 80000
)
SELECT department, COUNT(*) AS count
FROM high_earners
GROUP BY department
""")
result.show()
```

CTE Reference Table

Concept	SQL Query	PySpark Equivalent
CTE (Common Table Expressions)	WITH cte1 AS (SELECT * FROM table1) SELECT * FROM cte1 WHERE condition;	df.createOrReplaceTempView("cte1"); df_cte1 = spark.sql("SELECT * FROM cte1 WHERE condition")

Table 14: Common Table Expressions usage

12. Set Operations

UNION

SQL:

```
SELECT name FROM employees_2023
UNION
SELECT name FROM employees_2024;
```

PySpark DataFrame API:

```
df_2023.select("name").union(df_2024.select("name")).show()
```

UNION ALL

SQL:

```
SELECT name FROM employees_2023
UNION ALL
SELECT name FROM employees_2024;
```

PySpark DataFrame API:

```
df_2023.select("name").unionAll(df_2024.select("name")).show()
```


INTERSECT

SQL:
SELECT name FROM employees_2023
INTERSECT
SELECT name FROM employees_2024;

PySpark DataFrame API:
df_2023.select("name").intersect(df_2024.select("name")).show()

EXCEPT (Difference)

SQL:
SELECT name FROM employees_2023
EXCEPT
SELECT name FROM employees_2024;

PySpark DataFrame API:
df_2023.select("name").subtract(df_2024.select("name")).show()

Set Operations Reference

Concept	SQL Query	PySpark Equivalent
UNION	SELECT column FROM table1 UNION SELECT column FROM table2;	df1.union(df2).select("column")
UNION ALL	SELECT column FROM table1 UNION ALL SELECT column FROM table2;	df1.unionAll(df2).select("column")

Table 15: Set operations comparison

Join, Grouping and Pivoting Operations

Conce pt	SQL Query	PySpark Equivalent
JOIN	SELECT * FROM table1 JOIN table2 ON table1.column = table2.column;	df1.join(df2, "column")

GROUP BY	SELECT column, agg_function(column) FROM table GROUP BY column;	df.groupBy("column").agg(agg_function("column"))
PIVOT	PIVOT (agg_function(column) FOR pivot_column IN (values));	df.groupBy("pivot_column").pivot("column").agg(agg_function)

Table 16: Join, grouping, and pivot operations

13. Data Modification (DDL/DML)

Creating Tables

SQL:

```
CREATE TABLE employees (
  id INT,
  name STRING,
  department STRING,
  salary DOUBLE
);
```

PySpark DataFrame API:

```
from pyspark.sql.types import StructType, StructField, IntegerType, StringType, DoubleType
```

```
schema = StructType([
  StructField("id", IntegerType(), True),
  StructField("name", StringType(), True),
  StructField("department", StringType(), True),
  StructField("salary", DoubleType(), True)
])
```

```
df = spark.createDataFrame([], schema)
df.write.saveAsTable("employees")
```

INSERT Data

SQL:

```
INSERT INTO employees VALUES (1, 'John Doe', 'IT', 75000);
```

PySpark DataFrame API:

```
new_data = [(1, 'John Doe', 'IT', 75000)]
new_df = spark.createDataFrame(new_data, ["id", "name", "department", "salary"])
new_df.write.mode("append").saveAsTable("employees")
```

UPDATE (Not directly supported in standard PySpark)

SQL:

```
UPDATE employees  
SET salary = salary * 1.1  
WHERE department = 'IT';
```

PySpark DataFrame API (using Delta Lake or recreating DataFrame):

```
from pyspark.sql.functions import when
```

```
df = df.withColumn("salary",  
when(df.department == "IT", df.salary * 1.1)  
.otherwise(df.salary)  
)
```

DELETE (Not directly supported in standard PySpark)

SQL:

```
DELETE FROM employees WHERE age > 65;
```

PySpark DataFrame API:

```
df = df.filter(df.age <= 65)
```

14. Combining SQL and DataFrame APIs

PySpark allows seamless integration between SQL and DataFrame APIs[4]. You can execute SQL queries on DataFrames and vice versa.

DataFrame to SQL

Create temporary view from DataFrame:

```
df.createOrReplaceTempView("employees")
```

Now run SQL queries

```
result = spark.sql("SELECT * FROM employees WHERE salary > 50000")  
result.show()
```

SQL to DataFrame

SQL query returns a DataFrame:

```
sql_df = spark.sql("""  
SELECT department, AVG(salary) AS avg_salary  
FROM employees  
GROUP BY department  
""")
```


Continue with DataFrame operations

```
sql_df.filter(sql_df.avg_salary > 60000).show()
```

15. Performance Optimization Tips

Concept	SQL	PySpark
Caching	CACHE TABLE employees	df.cache() or df.persist()
Partitioning	Use partitioned tables in Hive	df.repartition("column")
Broadcasting	Automatic for small tables	broadcast(small_df) in joins
Query Optimization	Catalyst optimizer handles automatically	Catalyst optimizer handles automatically
Avoiding Shuffles	Reduce JOINS and GROUP BYs	Use repartition() strategically

Table 17: Performance optimization comparison

Summary Comparison Table

Operation	SQL	PySpark Method
Select	SELECT	.select()
Filter	WHERE	.filter() or .where()
Group	GROUP BY	.groupBy()
Sort	ORDER BY	.orderBy()
Join	JOIN	.join()
Aggregate	COUNT, AVG, SUM	.agg(count(), avg(), sum())
Distinct	DISTINCT	.distinct()
Limit	LIMIT n	.limit(n)
Union	UNION	.union()

Table 18: Quick reference: SQL to PySpark mapping

Key Takeaways

- **SQL is declarative** – You describe what you want; the database figures out how to get it
- **PySpark is procedural** – You define the steps to transform data programmatically
- **Performance is comparable** – Both use Spark's Catalyst Optimizer for query optimization[5]
- **Choose SQL when:** You're more comfortable with database queries, need simple analytics, or working with non-programmers
- **Choose PySpark when:** You need complex transformations, programmatic control, integration with Python libraries, or building production pipelines
- **Best practice:** Master both approaches – use SQL for ad-hoc queries and PySpark DataFrame API for complex ETL pipelines
- **Interoperability:** You can mix both approaches in the same application by converting DataFrames to SQL views and vice versa[6]

Conclusion

Both SQL and PySpark are essential tools in the modern data engineer's toolkit. SQL provides familiar, readable syntax for data analysts and database professionals, while PySpark offers programmatic flexibility and scalability for distributed big data processing.

Understanding both approaches allows you to choose the right tool for each task and leverage the strengths of each paradigm. In practice, many data engineering workflows combine SQL for exploratory analysis and PySpark DataFrames for production ETL pipelines.

References

- [1] Soutir Sen. (2025, February 8). PySpark vs SQL Syntax Breakdown. *Substack*.
<https://substack.com/pyspark-vs-sql>
- [2] Seequality. (2024, March 23). Pyspark – cheatsheet with comparison to SQL.
<https://pl.seequality.net/pyspark-cheatsheet-comparison-sql>
- [3] Towards Data Science. (2025, January 19). SQL to PySpark. <https://towardsdatascience.com/sql-to-pyspark>
- [4] SparkByExamples. (2025, April 8). PySpark SQL vs DataFrames: What's the Difference?
<https://sparkbyexamples.com/pyspark/pyspark-sql-vs-dataframes>
- [5] LinkedIn. (2024, April 30). Difference between SQL and PySpark.
<https://www.linkedin.com/pulse/difference-between-sql-pyspark>
- [6] Dataquest. (2025, October 20). Using Spark SQL in PySpark for Distributed Data Analysis.
<https://www.dataquest.io/blog/spark-sql-pyspark>

[7] Various Authors. (2024). SQL & PySpark Equivalence: A Comprehensive Guide. *Technical Documentation*.

Prahalad's Learning